

Szczegółowy opis parsowania strumienia TS i PES

1. Parsowanie Transport Stream (TS)

Transport Stream to kontener, który składa się z pakietów o stałej długości (188 bajtów). Każdy pakiet TS zawiera:

1.1. Nagłówek TS (TS Header)

```
TS_PacketHeader.Parse(TS_PacketBuffer);
```

W nagłówku znajdują się kluczowe informacje:

- Bajt synchronizacji (sync byte = 0x47) - oznacza początek pakietu
- PID (Program ID) - identyfikator strumienia
- Flagi kontrolne (np. czy pakiet zawiera pole adaptacyjne)

1.2. Pole adaptacyjne (Adaptation Field) - opcjonalne

```
if (TS_PacketHeader.hasAdaptationField()) {  
    TS_PacketAdaptationField.Parse(  
        TS_PacketBuffer + xTS::TS_HeaderLength,  
        TS_PacketHeader.getAdaptationFieldControl());  
}
```

Pole adaptacyjne może zawierać:

- Informacje o PCR (Program Clock Reference)
- Flagi nieciągłości
- Wypełnienie (padding)

1.3. Dane użytkowe (Payload)

- Zawierają fragmenty pakietów PES

2. Parsowanie PES (Packetized Elementary Stream)

PES to warstwa, która zawiera właściwe dane (audio/video). Proces składania pakietów PES jest bardziej złożony:

2.1. Inicjalizacja Assemblera

```
xPES_Assembler PES_Assembler;  
PES_Assembler.Init(pid);
```

2.2. Składanie pakietów PES

```
auto Result = PES_Assembler.AbsorbPacket(  
    TS_PacketBuffer,  
    &TS_PacketHeader,  
    &TS_PacketAdaptationField);
```

2.3. Stany składania PES

a) AssemblingStarted - znaleziono początek nowego pakietu PES

```
case xPES_Assembler::eResult::AssemblingStarted:  
    // Znaleziono nagłówek PES  
    // Rozpoczyna się zbieranie danych
```

- Wykryto znacznik początku pakietu PES
- Parsowany jest nagłówek PES
- Rozpoczyna się zbieranie danych

b) AssemblingContinue - kontynuacja zbierania danych

```
case xPES_Assembler::eResult::AssemblingContinue:  
    // Kolejne dane są dodawane do bufora
```

- Dane z kolejnych pakietów TS są dodawane do bufora PES
- Trwa akumulacja danych

c) AssemblingFinished - zakończono składanie pakietu PES

```

case xPES_Assembler::eResult::AssemblingFinished: {
    int headLen = PES_Assembler.getPESHeaderLength();
    int dataLen = PES_Assembler.getNumPacketBytes();
    int pktLen = PES_Assembler.getPESPacketLength();

    // Zapisz dane payload
    const uint8_t* payload = PES_Assembler.getPayloadBuffer();
    // Zapisz dane pomijając nagłówek PES
    fwrite(payload + headLen, 1, dataLen - headLen, output_file);
}

```

- Pakiet PES jest kompletny
- Można wyodrębnić właściwe dane (payload)
- Dane są zapisywane do pliku wyjściowego

d) StreamPacketLost - wykryto utratę pakietu

```

case xPES_Assembler::eResult::StreamPackedLost:
    // Wykryto brak ciągłości w strumieniu

```

- Wykryto nieciągłość w numeracji pakietów
- Może oznaczać utratę danych

3. Struktura danych

3.1. Pakiet TS (188 bajtów)

```

[Sync (1B)][Header (3B)][Adaptation Field (optional)][Payload]

```

3.2. Pakiet PES

```

[Start Code (3B)][Stream ID (1B)][Length (2B)][Header][Payload]

```

4. Proces przetwarzania

1. Czytanie pakietu TS (188 bajtów)
2. Sprawdzenie bajtu synchronizacji (0x47)
3. Parsowanie nagłówka TS i pobranie PID
4. Jeśli jest pole adaptacyjne - jego parsowanie

5. Przekazanie danych do odpowiedniego assemblera PES

6. Składanie pakietu PES:

- o Wykrycie początku pakietu PES
- o Zbieranie danych z kolejnych pakietów TS
- o Zakończenie składania gdy pakiet jest kompletny

7. Wyodrębnienie i zapisanie danych użytkowych

Ten proces pozwala na wyodrębnienie z multipleksowanego strumienia transportowego pojedynczych strumieni elementarnych (audio, video, dane), które mogą być następnie dekodowane przez odpowiednie dekodery.